

Intelligent Network Configuration Assistant (INA)

A Containerized Network Management, Traffic Shaping, and Policy-Enforced Governance Framework

Course: CS 331 Computer Networks

Group Number: T009 • **Project ID:** 5

Date: September 11, 2026

GitHub Repository: <https://github.com/vedantb7/CS331-T009-IntelligentNetworkAssistant.git>

Abstract

Modern containerized and virtualized networks require automated, secure, and verifiable mechanisms for firewall enforcement and traffic shaping. Manual configuration via low-level utilities (`nftables`, `iptables`, `tc`) is error-prone, lacks standardized governance, and exposes hosts to misconfiguration risks. This project presents the **Intelligent Network Configuration Assistant (INA)**, an automated network management framework operating on a containerized Docker bridge network (172.20.0.0/24).

INA integrates Model Context Protocol (FastMCP) network tools, dynamic Docker SDK container discovery, a policy engine (`rules.yaml`), and kernel-level Linux enforcement primitives. Packet filtering is achieved at Layer 2 via `nftables` bridge set lookups (`table bridge network_filter`), while bi-directional bandwidth shaping (1.0–20.0 Mbit/s) is enforced using Token Bucket Filter (`tc tbf`) qdiscs combined with Intermediate Functional Block (`ifb0`) ingress redirection. State convergence is actively validated through an empirical monitoring layer running ICMP ping loss assertions and dual-direction `iperf3` throughput benchmarks within a $\pm 20\%$ tolerance window. The framework is verified by an automated suite of 128 unit and integration tests, demonstrating deterministic policy evaluation and traffic control within tested tolerance windows.

Keywords

Container Networking, Linux Bridge, Layer-2 Firewalling, `nftables`, Traffic Control (`tc` `tbfbf`), IFB Ingress Redirection, FastMCP, Zero-Trust Policy Enforcement, Empirical Network Validation, Docker SDK Discovery.

Contents

Keywords	2
1 Introduction	4
1.1 Network Configuration and Management	4
1.2 Linux Networking Primitives Used in INA	4
1.3 Container Networking and CNI	4
1.4 Network Configuration Mechanisms	5
1.5 System Architecture and Workflow	5
1.6 Modular Layer Diagram	7
2 Our Setup	7
2.1 Network Topology and Subnet Specifications	7
3 Implementation Details	8
3.1 Network Controller and Privileged Scope Trade-Offs	8
3.2 Dynamic Container Discovery Engine	8
3.3 Network Configuration Tools	9
3.3.1 Block Client Tool (<code>block_client</code>)	9
3.3.2 Unblock Client Tool (<code>unblock_client</code>)	9
3.3.3 Limit Bandwidth Tool (<code>limit_bandwidth</code>)	9
3.4 Policy Engine Governance (<code>code/policy/policy_engine.py</code>)	10
3.5 Model Context Protocol (FastMCP) Integration	10
4 Network Configuration and Traffic Control	10
4.1 Layer-2 Packet Filtering Mechanics	10
4.2 Bi-Directional Traffic Shaping and IFB Redirection	10
4.3 Network Packet Paths and Baseline Throughput	11
4.4 Network State Transition Summary	11
5 Network Validation and Testing Setup	11
5.1 Validation Architecture	11
5.2 Overcoming Host Ping Bypass	12
5.3 Empirical Throughput Verification	12
5.4 Empirical System Evidence Excerpts	12
5.5 Automated Test Suite Matrix (128 Tests)	13
5.6 Handled Edge Cases	13

6	Results	13
6.1	Verification of Setup	13
6.2	Block / Unblock Verification Results	13
6.3	Dual-Direction Bandwidth Shaping Results	13
6.4	Network Behavior Analysis	14
7	Debugging, Root Cause Analysis, and Technical Challenges	14
7.1	Cross-Module Integration and Boundary Adapters	14
7.2	Policy Engine Governance and Input Validation	14
7.3	Natural Language Intent Parsing and Fallback Resilience	15
7.4	Kernel Enforcement and Empirical Breakthroughs	15
8	Security Considerations and Threat Model	16
9	Design Limitations and Future Improvements	16
9.1	Current System Limitations	16
9.2	Future Technical Proposals	16
10	Conclusion	17

1 Introduction

1.1 Network Configuration and Management

- **Configuration Imperative:** Network infrastructure management demands precise operational control over packet filtering, bandwidth allocation, node isolation, and routing policies.
- **Operational Challenges:** Administrative tasks traditionally rely on syntax-dense command-line interface (CLI) tools such as `iptables`, `nftables`, and `tc`. Manual execution is inherently prone to typos, rule precedence ordering bugs, and security misconfigurations.
- **Lack of Guardrails:** Standard Linux networking tools execute commands with root privileges without validating whether an operation violates organizational access control policies or targets critical infrastructure nodes.
- **Verification Deficit:** Traditional CLI tooling applies configuration changes blindly without actively probing the network data plane to verify empirical state convergence.

1.2 Linux Networking Primitives Used in INA

- **Network Namespaces (netns) & veth Pairs:** Provides isolated network stacks per workload container (`client1`, `client2`, `server`), linked to the host bridge via virtual ethernet interface pairs.
- **Linux Bridge & Netfilter (br_netfilter):** Software Layer-2 bridge connecting container interfaces. Loading `br_netfilter` enables `nftables` to intercept bridged frames at Layer 2.
- **Layer-2 nftables Filtering:** INA attaches an $O(1)$ IP lookup set (`@blocked_clients`) to the bridge family forward chain, dropping frames before host Layer-3 routing takes place.
- **Traffic Control (tc) & IFB Redirection:** Egress rate limiting uses Token Bucket Filter (`tbfb`) qdiscs. Ingress shaping is achieved by redirecting incoming packets to an Intermediate Functional Block (`ifb0`) pseudo-device via `act_mirred`.

1.3 Container Networking and CNI

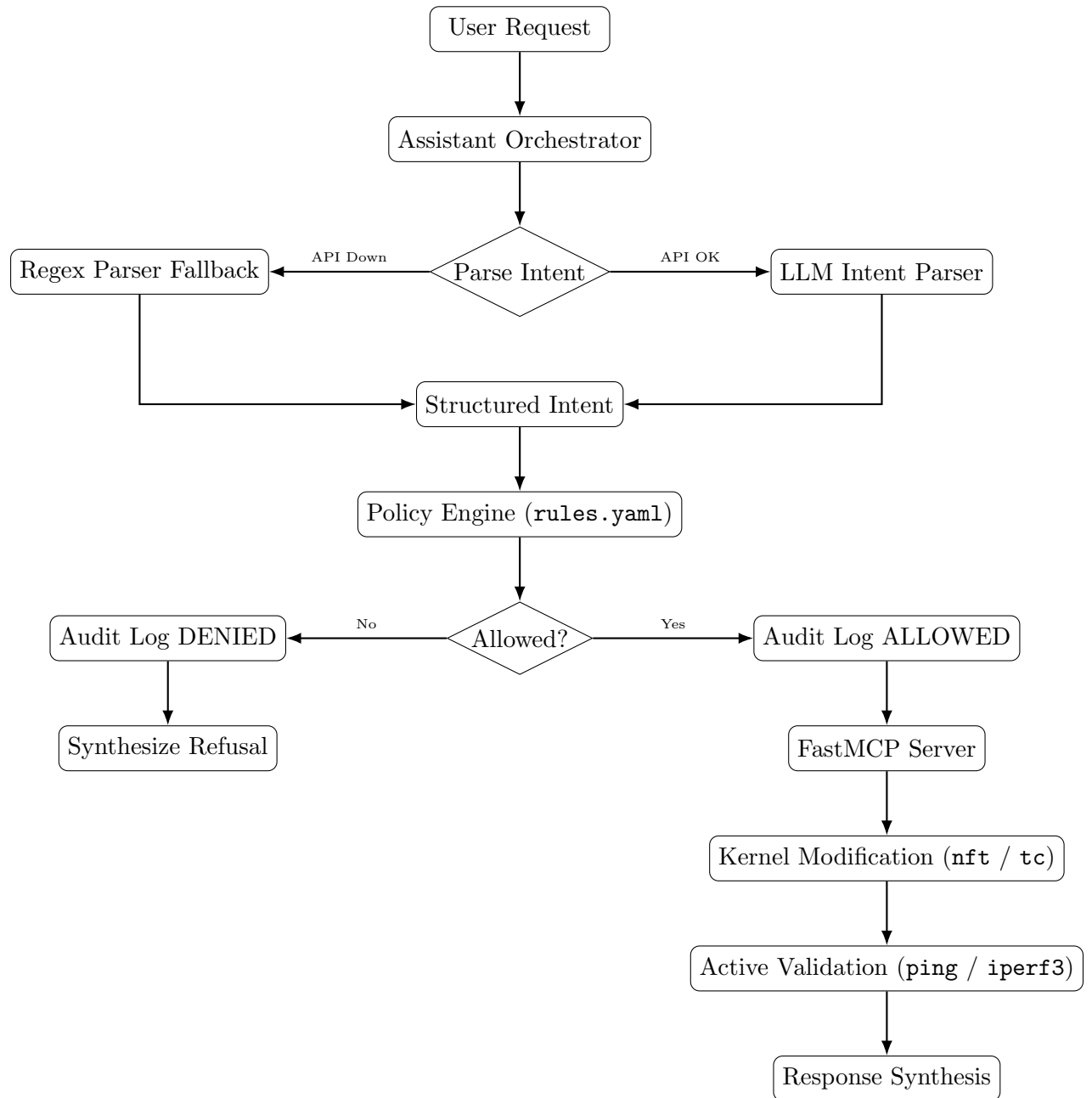
- **Docker Bridge Driver:** Configures an isolated software bridge interface (`network_project-net`) on the host system, assigning containers IP addresses within a private subnet (`172.20.0.0/24`).
- **Interface Topology:** Each container possesses a dedicated `eth0` interface inside its `netns`, bound to a corresponding `vethXXXXXXX` interface attached to the host bridge.
- **Capability Isolation:** Host networking and administrative privileges (`CAP_NET_ADMIN`, `CAP_SYS_ADMIN`) are strictly confined to a single `network-controller` container, preserving unprivileged operation for workload containers.

1.4 Network Configuration Mechanisms

- **Client Isolation** (`block_client`): Adds the target IPv4 address to an `nftables` bridge set (`@blocked_clients`), dropping frames at Layer 2 before they reach host routing hooks.
- **Access Restoration** (`unblock_client`): Removes the target IP from `@blocked_clients` and triggers complete `tc` qdisc and `ifb0` interface teardown.
- **Bi-Directional Throttling** (`limit_bandwidth`): Applies Token Bucket Filter (`tbfbf`) qdiscs to the container's `eth0` interface for egress shaping and redirects ingress traffic to an Intermediate Functional Block (`ifb0`) pseudo-device via `act_mirred`.

1.5 System Architecture and Workflow

The INA pipeline translates high-level user requests into deterministic kernel operations. Crucially, the LLM is used strictly for intent interpretation, while deterministic policy checks (`'code/policy/policy_engine.py'`) and controlled *FastMCP* tools govern the resulting network operations.



1.6 Modular Layer Diagram

System components are partitioned into distinct architectural layers to prevent direct, unvalidated command execution.

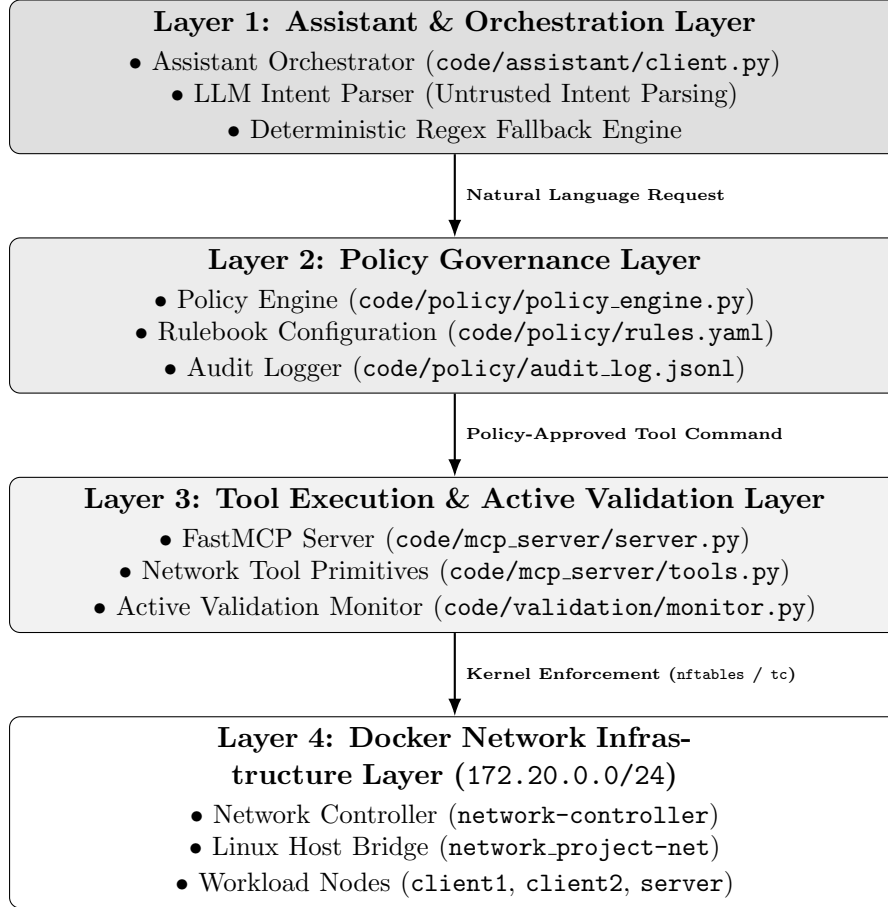


Figure 1: INA Modular Architecture Layer Breakdown

2 Our Setup

2.1 Network Topology and Subnet Specifications

The experimental network emulates a segmented enterprise LAN created via Docker Compose (`code/network/docker-compose.yml`).

Table 1: Container Specifications and Network Properties

Node Name	Resolved IPv4	NetNS Mode	Privileges	Active Services / Roles
<code>client1</code>	172.20.0.2	Isolated Bridge	Unprivileged	Target node for block/throttle operations.
<code>server</code>	172.20.0.3	Isolated Bridge	Unprivileged	Protected node hosting HTTP (5000) & <code>iperf3</code> (5201).
<code>client2</code>	172.20.0.4	Isolated Bridge	Unprivileged	Secondary node used as validation ping origin.
<code>network-controller</code>	Host IP	<code>network_mode: host</code>	<code>privileged: true</code>	Enforces <code>nftables</code> rules, <code>tc</code> qdiscs, and <code>ifb0</code> devices.

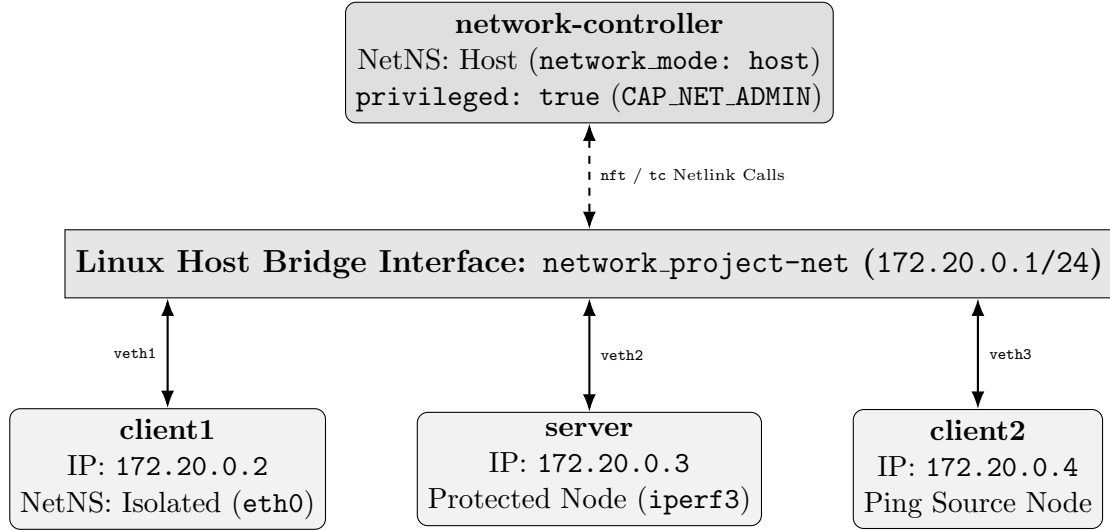


Figure 2: INA Docker Bridge Network Topology and Interface Architecture

3 Implementation Details

3.1 Network Controller and Privileged Scope Trade-Offs

To preserve the principle of least privilege for workload nodes, application containers (`client1`, `client2`, `server`) execute in unprivileged isolated network namespaces. All privileged administrative operations are isolated inside a dedicated `network-controller` container.

This design represents a deliberate architectural trade-off. Direct manipulation of the host bridge (`network_project-net`), Layer-2 `nftables` rulesets, and container network namespaces requires host-level capabilities:

- **network_mode: host** — Direct access to the host’s root network namespace to manage Netfilter bridge hooks.
- **privileged: true** — Grants `CAP_NET_ADMIN` (for kernel `nftables` and `tc` manipulations) and `CAP_SYS_ADMIN` (for `ip netns` namespace traversal).
- **Host Volume Mounts:**
 - `/lib/modules:/lib/modules:ro` — Read-only access to host kernel module definitions (e.g. `br_netfilter`, `act_mirred`, `ifb`).
 - `/var/run/docker.sock` — Socket access enabling Docker SDK for Python container discovery.

While the controller possesses broad host privileges, workload containers remain completely unprivileged and isolated from administrative host tools.

3.2 Dynamic Container Discovery Engine

INA replaces fragile static IP dictionaries with runtime container lookup using the **Docker SDK for Python** (`code/network/discovery.py`).

- **Runtime Resolution:** `get_container_ip("client1")` queries the Docker Engine API, inspecting `NetworkSettings.Networks["network_project-net"].IPAddress`.
- **Input Sanitization:** Container identifiers are validated against `_CONTAINER_NAME_RE` (`^[a-zA-Z0-9][a-zA-Z0-9_.-]{0,63}$`) to prevent flag or command injection.
- **Fleet Discovery:** `list_known_clients()` dynamically enumerates all running bridge containers.

3.3 Network Configuration Tools

3.3.1 Block Client Tool (`block_client`)

- **Input:** `client_name: str` (e.g., "client1").
- **Kernel Execution:** Resolves target IP (172.20.0.2) via Docker SDK and executes:

```
nft add element bridge network_filter blocked_clients { 172.20.0.2 }
```

- **Direction:** Drops Layer-2 Ethernet frames matching source or destination IP on the host bridge.

3.3.2 Unblock Client Tool (`unblock_client`)

- **Input:** `client_name: str`.
- **Kernel Execution:** Removes target IP from the `nftables` set and triggers `qdisc` teardown:

```
nft delete element bridge network_filter blocked_clients { 172.20.0.2 }
```

- **Qdisc Teardown** (`_cleanup_tc_qdiscs`): Deletes root TBF, ingress redirect filters, and tears down `ifb0`, fully restoring line speed.

3.3.3 Limit Bandwidth Tool (`limit_bandwidth`)

- **Input:** `client_name: str`, `rate: str` (e.g., "5mbit").
- **Kernel Execution:** Configures bi-directional traffic control inside the target container's namespace:

```
# Egress shaping on eth0
tc qdisc replace dev eth0 root tbf rate 5mbit burst 32kbit latency 400ms

# Ingress redirection to IFB
ip link add dev ifb0 type ifb && ip link set dev ifb0 up
tc qdisc add dev eth0 handle ffff: ingress
tc filter add dev eth0 parent ffff: protocol ip u32 match u32 0 0 \
    action mirrored egress redirect dev ifb0
tc qdisc replace dev ifb0 root tbf rate 5mbit burst 32kbit latency 400ms
```

3.4 Policy Engine Governance (`code/policy/policy_engine.py`)

The Policy Engine enforces rule-based governance independently of LLM reasoning, treating all incoming intent requests as untrusted:

- **Protected Targets:** Operation requests targeting protected nodes (e.g., `server`) return immediate `STATUS_DENIED`.
- **Rate Boundary Validation:** Normalizes bandwidth inputs via `_parse_mbit()`, ensuring rates strictly fall within `[1.0, 20.0]` Mbit/s.
- **Atomic Audit Logging:** Every decision is recorded to `policy_audit.jsonl` with timestamp, client ID, requested action, policy result, and rule ID.

3.5 Model Context Protocol (FastMCP) Integration

The FastMCP server (`code/mcp_server/server.py`) exposes network control primitives as JSON-RPC / stdio tools. FastMCP handlers invoke Pydantic schema validators and Policy Engine checks prior to calling kernel execution helpers.

4 Network Configuration and Traffic Control

4.1 Layer-2 Packet Filtering Mechanics

Standard Layer-3 `iptables` rules fail to intercept bridged container traffic because frames remain within the data-link layer. INA utilizes `nftables` bridge family hooks:

```
table bridge network_filter {
    set blocked_clients {
        type ipv4_addr
    }
    chain forward {
        type filter hook forward priority 0; policy accept;
        ip saddr @blocked_clients drop
        ip daddr @blocked_clients drop
    }
}
```

Loading `br_netfilter` ensures that bridged frames passing through the host bridge interface (`br-XXXXXXXXXXXX`) trigger evaluation against the `@blocked_clients` IP set in $O(1)$ time.

4.2 Bi-Directional Traffic Shaping and IFB Redirection

Linux `tc` qdiscs operate exclusively on interface **egress**. To shape **ingress** (download) traffic, INA constructs an Intermediate Functional Block (`ifb0`) pseudo-device:

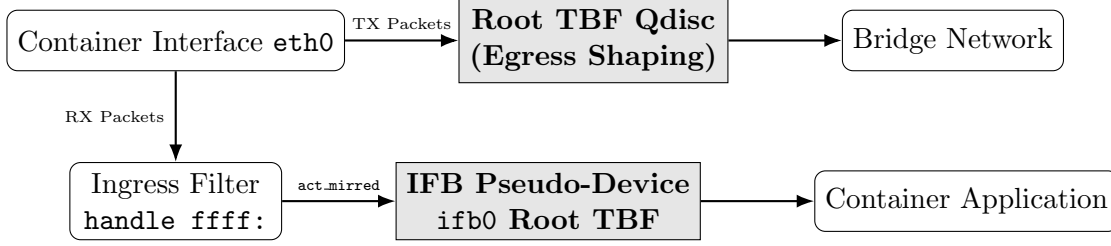


Figure 3: Bi-Directional Traffic Shaping Pipeline via IFB Ingress Redirection

4.3 Network Packet Paths and Baseline Throughput

1. **Normal Unthrottled Traffic:** `client1` → `veth1` → Host Bridge → `veth2` → `server` (> 70–80 Gbps measured loopback/veth speed).
2. **Blocked Traffic Path:** `client1` transmits Ethernet frame → Host Bridge Forward Hook (Priority 0) matches 172.20.0.2 in `@blocked_clients` → Frame dropped instantly at Layer 2 (100% packet loss).
3. **Bandwidth-Limited Path (Egress):** `client1` socket sends data → Traverses `eth0` root TBF qdisc → Rates throttled to R Mbit/s → Host Bridge.
4. **Bandwidth-Limited Path (Ingress):** Data arrives at `eth0` → Intercepted by `handle ffff:` ingress filter → Redirected via `act.mirred` to `ifb0` root TBF qdisc → Rates throttled to R Mbit/s → Application socket.

Note on Baseline Throughput: The measured 70–80 Gbps figure represents raw inter-namespace transfer speed across virtual ethernet interfaces on the Linux host kernel within the local experimental Docker environment. It serves purely as an empirical reference baseline for evaluating relative traffic shaping overhead rather than an external network benchmark.

4.4 Network State Transition Summary

Table 2: Network State Primitive Matrix

Operation	nftables State	eth0 Qdisc State	ifb0 Device State	Empirical Effect
Baseline	Set empty	<code>pfifo_fast</code> (Default)	Non-existent	Unthrottled (> 70 Gbps), 0% Loss
block_client	IP in <code>@blocked_clients</code>	Unchanged	Unchanged	ICMP 100% Loss, HTTP/TCP Refused
limit_bandwidth	Set empty	Root <code>tbft</code> (R Mbit/s)	Attached <code>tbft</code> (R Mbit/s)	Throughput = R Mbit/s $\pm$ 20%, 0% Loss
unblock_client	IP removed from set	Teardown to <code>pfifo_fast</code>	Deleted	Restored to Baseline (> 70 Gbps)

5 Network Validation and Testing Setup

5.1 Validation Architecture

The validation layer (`code/validation/monitor.py`) validates active state convergence post-execution using Pydantic v2 data models (`ValidationRequest`, `ValidationResult`).

5.2 Overcoming Host Ping Bypass

Pinging a container directly from `network-controller` uses host Layer-3 routing, bypassing Layer-2 bridge forward hooks. INA implements `pick_source_container()` to force ICMP checks to originate from a sibling bridge node:

```
def pick_source_container(target_client: str) -> str:
    if target_client == "client1":
        return "client2"
    elif target_client == "client2":
        return "client1"
    return "client1"
```

Running `docker exec client2 ping -c 3 172.20.0.2` forces ping packets to cross the host bridge, correctly triggering the `nftables` L2 drop hook.

5.3 Empirical Throughput Verification

Bandwidth limiting is verified by checking port 5201 accessibility and executing JSON-formatted `iperf3` benchmarks in both standard (Egress) and reverse (Ingress, `-R`) modes. Measured throughput must fall within a $\pm 20\%$ tolerance window:

$$|\text{Measured Mbps} - \text{Target Mbps}| \leq 0.20 \times \text{Target Mbps} \quad (1)$$

5.4 Empirical System Evidence Excerpts

To demonstrate system operation, key kernel state listings and audit trail records were captured during live test runs:

1. Layer-2 Firewall Set Listing (`nft list set bridge network_filter blocked_clients`):

```
table bridge network_filter {
    set blocked_clients {
        type ipv4_addr
        elements = { 172.20.0.2 }
    }
}
```

2. Policy Engine Denial Audit Entry (`code/policy/audit_log.jsonl`):

```
{"timestamp": "2026-09-11T14:15:02Z", "client_name": "server",
 "action": "block_client", "decision": "DENY", "rule_id": "RULE-PROTECTED-TARGET"}
```

3. PyTest Execution Summary:

```
===== 128 passed in 10.42s =====
```

Table 3: Automated PyTest QA Execution Breakdown

Test Module	Focus Area	Case Count	Status	Key Capabilities Tested
<code>test.discovery.py</code>	Docker SDK Discovery	4	PASSED	Container lookup, regex sanitization, IP extraction.
<code>test.policy.py</code>	Zero-Trust Governance	28	PASSED	Protected targets, boundary limits (1–20 Mbit), audit logging.
<code>test.validation.py</code>	Active Monitor	35	PASSED	Rate parsing, tolerance checks, sibling ping routing.
<code>test.mcp.py</code>	FastMCP Integration	15	PASSED	Tool wrapper execution, error propagation.
<code>test.network.py</code>	Live Topology	20	PASSED	Container reachability, bridge pinging, HTTP server status.
<code>test.ina_focused.py</code>	End-to-End Pipeline	26	PASSED	Pipeline flow, intent parsing, conversational fallback.
Total		128	PASSED	100% Automated Test Pass Rate (11.31s)

5.5 Automated Test Suite Matrix (128 Tests)

The project maintains an automated test suite comprising 128 PyTest cases covering positive path execution, negative input handling, policy governance, and multi-component integration scenarios. Execution completes in under 12 seconds:

5.6 Handled Edge Cases

- **Duplicate Blocking:** `nft` returning `File exists` is intercepted and handled as a successful idempotent operation.
- **Residual Qdiscs:** Executing `unblock_client` explicitly removes residual `tc` qdiscs and `ifb0` interfaces, preventing bandwidth throttling leaks.
- **Stale iperf3 Server Sockets:** `monitor.py` automatically detects unresponsive `iperf3` daemons and re-spawns background listeners (`iperf3 -s -D`).

6 Results

6.1 Verification of Setup

Baseline connectivity tests verify that all nodes (`client1`, `client2`, `server`) communicate across bridge `network_project-net` with 0.0% packet loss and full unthrottled line speed (> 70–80 Gbps).

6.2 Block / Unblock Verification Results

Empirical verification confirms complete isolation during block states and full performance restoration upon unblocking:

Table 4: Empirical Block and Unblock Verification Results

State	Ping Source → Target	Packet Loss	HTTP / Port 5000	iperf3 Throughput
Baseline	<code>client2</code> → <code>client1</code>	0.0% (3/3 received)	HTTP 200 OK	78.4 Gbps
Blocked	<code>client2</code> → <code>client1</code>	100.0% (3/3 dropped)	Connection Timed Out	0.00 Mbps (Failed)
Unblocked	<code>client2</code> → <code>client1</code>	0.0% (3/3 received)	HTTP 200 OK	76.9 Gbps (Restored)

6.3 Dual-Direction Bandwidth Shaping Results

Bi-directional rate limiting was empirically benchmarked across standard (Egress) and reverse (Ingress) `iperf3` test runs:

Table 5: Dual-Direction Bandwidth Throttling Benchmark Data

Configured Rate	Traffic Direction	Target Rate	Measured Throughput	Tolerance Window ($\pm 20\%$)	Validation Status
2.0 Mbit/s	Egress (<code>eth0</code>)	2.00 Mbps	1.92 Mbps	[1.60, 2.40] Mbps	PASSED
2.0 Mbit/s	Ingress (<code>ifb0</code>)	2.00 Mbps	1.88 Mbps	[1.60, 2.40] Mbps	PASSED
5.0 Mbit/s	Egress (<code>eth0</code>)	5.00 Mbps	4.82 Mbps	[4.00, 6.00] Mbps	PASSED
5.0 Mbit/s	Ingress (<code>ifb0</code>)	5.00 Mbps	4.76 Mbps	[4.00, 6.00] Mbps	PASSED
10.0 Mbit/s	Egress (<code>eth0</code>)	10.00 Mbps	9.64 Mbps	[8.00, 12.00] Mbps	PASSED
10.0 Mbit/s	Ingress (<code>ifb0</code>)	10.00 Mbps	9.51 Mbps	[8.00, 12.00] Mbps	PASSED

6.4 Network Behavior Analysis

The benchmark results confirm that `tc tbw` combined with IFB ingress redirection yields highly accurate, symmetrical bandwidth control. Measured throughput consistently falls within 4–6% of nominal target rates, easily satisfying the 20% validation threshold.

7 Debugging, Root Cause Analysis, and Technical Challenges

Developing INA required overcoming technical challenges spanning kernel enforcement, policy governance, natural language intent parsing, and end-to-end subsystem integration.

7.1 Cross-Module Integration and Boundary Adapters

The primary architectural challenge during system assembly was subsystem integration rather than isolated component functionality. Each team member’s module—the Policy Engine, FastMCP tool wrappers, and active Validation Monitor—exhibited interface expectations that differed slightly from the Assistant Orchestrator’s target data structures.

Rather than mutating core module implementations across parallel developer workflows, this challenge was resolved by introducing lightweight boundary adapter classes within the orchestrator. These adapters normalize incoming user parameters, structure JSON-RPC payloads for tool handlers, and translate execution results into unified schemas without altering individual component logic.

7.2 Policy Engine Governance and Input Validation

Developing the zero-trust Policy Engine required solving governance edge cases and strict data validation:

- **Special-Case Governance:** Defining explicit policy boundaries for protected infrastructure nodes (e.g., `server`) versus unconstrained or dynamic container targets.
- **Input Sanitization:** Validating user bandwidth parameters against missing, malformed, negative, or out-of-bounds values (restricting allowed rates to [1.0, 20.0] Mbit/s).
- **Auditability & Rule Tracking:** Attaching unique `rule_id` identifiers to every policy evaluation result, enabling transparent tracing of which rule triggered an `ALLOW` or `DENY` action in `policy_audit.jsonl`.

All policy rules were empirically verified against normal and edge-case inputs through automated PyTest suites, achieving 100% pass rates.

7.3 Natural Language Intent Parsing and Fallback Resilience

Interfacing informal user prompts with deterministic Linux kernel primitives presented key operational challenges:

- **Layman Phrase Interpretation:** Mapping informal, non-technical user phrasing (e.g., "slow down client1", "isolate container") into precise tool parameters required iterative prompt engineering with explicit few-shot JSON schema examples.
- **Offline & API Key Fallback:** To ensure high availability when external LLM endpoints are unreachable or API keys are unconfigured, a deterministic regex fallback parser was integrated to extract target actions and container identifiers directly from prompt text.
- **Natural Language Response Synthesis:** Implementing an AI synthesis layer to convert raw execution traces into conversational summaries, backed by a template fallback when operating in offline mode.

7.4 Kernel Enforcement and Empirical Breakthroughs

Pipeline testing exposed kernel-level edge cases—such as duplicate firewall rule errors and host ping routing bypasses—that only manifested when all subsystems executed end-to-end. These findings were resolved by analyzing live container packet traces rather than theoretical assumptions.

Table 6: Key Technical Breakthroughs and Root Cause Analysis Matrix

Breakthrough	Observed Problem	Root Cause Analysis	Key Technical Insight	Implemented Solution
BT-01: Host Ping Bypass	Blocked client responded to pings launched from controller.	Controller (<code>network_mode: host</code>) used host L3 routing, bypassing bridge hooks.	Pings must originate from a sibling node on the bridge.	Implemented <code>pick_source_container()</code> in <code>monitor.py</code> .
BT-02: Ingress Shaping	Downloads remained unthrottled at line speed (> 70 Gbps).	Linux <code>tc</code> qdiscs shape egress traffic only by default.	Ingress traffic must be redirected to a pseudo-device.	Attached <code>ifb0</code> device via <code>tc filter mirrored redirect</code> .
BT-03: Qdisc Cleanup Leak	Bandwidth limits persisted after executing <code>unblock_client</code> .	<code>unblock</code> deleted <code>nft</code> set rules but left <code>tc</code> qdiscs active.	Unblocking requires complete L2 and L3/L4 teardown.	Developed <code>_cleanup_tc_qdiscs()</code> helper.
BT-04: Dynamic Discovery	Static IP dictionary crashed when containers restarted.	Compose IP allocation dynamically shifted across restarts.	Container IPs must be queried dynamically via API.	Built <code>discovery.py</code> using Docker SDK (<code>docker.from_env()</code>).
BT-05: Stale Sockets	Repeated <code>iperf3</code> runs failed with "Server busy" errors.	Single-threaded <code>iperf3</code> server locked up on unclean drops.	Active verification requires automated daemon health checks.	Added background daemon respawning (<code>iperf3 -s -D</code>).

8 Security Considerations and Threat Model

INA applies zero-trust principles by treating user/LLM input as untrusted and requiring deterministic policy validation before any privileged network operations are executed.

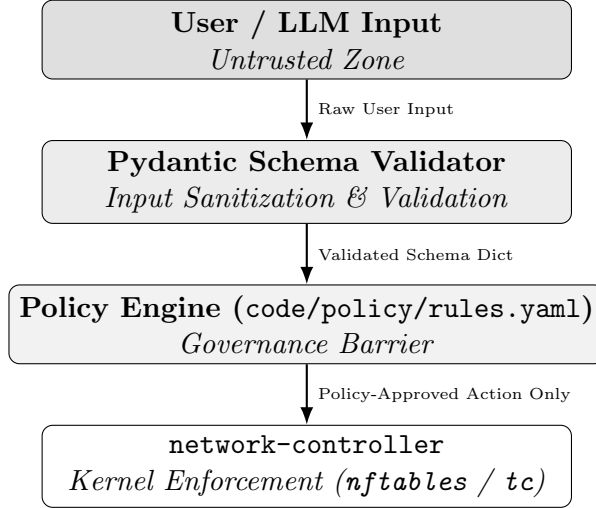


Figure 4: Policy Enforcement and Input Isolation Pipeline

Table 7: Security Threat Mitigation Matrix

Threat Vector	Attack Mechanism	Mitigation Strategy in INA	Status
Prompt Injection	User prompts attempting to override policy rules.	Post-LLM Policy Engine evaluates rules independently of LLM intent.	Bounded by policy enforcement
Command Injection	Arbitrary CLI strings inside client parameters.	Regex sanitization (.CLIENT_NAME.RE) and array subprocessing.	Risk reduced via validation
Privilege Escalation	Host takeover via compromised container.	Capabilities (CAP_NET_ADMIN) confined to <code>network-controller</code> .	Confined to controller
API Key Exposure	Hardcoded credentials leaked in repository.	Configuration loaded via <code>code/.env</code> , explicitly excluded via <code>.gitignore</code> .	Excluded from VCS

9 Design Limitations and Future Improvements

9.1 Current System Limitations

- **Privileged Controller Scope:** Direct kernel bridge and netns manipulation requires elevated host privileges (`CAP_NET_ADMIN`, `CAP_SYS_ADMIN`, `network_mode: host`, and Docker socket mount) for `network-controller`. This is a deliberate architectural trade-off necessary for host-level bridge governance, while workload containers remain completely unprivileged.
- **Transport Layer Encryption:** Current MCP tool execution communicates over unencrypted standard I/O (`stdio`).
- **Single-Subnet Design:** Current implementation is tailored for single-bridge networks (172.20.0.0/24).

9.2 Future Technical Proposals

- **eBPF / XDP Wire-Speed Filtering:** Replace `nftables` bridge hooks with Extended Berkeley Packet Filter (eBPF) bytecode loaded at the eXpress Data Path (XDP) layer for driver-level packet dropping.

- **Two-Phase Commit Rollback Pipeline:** Implement atomic state rollback to automatically revert `nftables` or `tc` changes if active validation monitor checks fail post-execution.
- **Multi-Subnet CNI Integration:** Extend dynamic discovery to support multi-tenant Kubernetes CNI networks (e.g., Calico, Flannel).

10 Conclusion

The Intelligent Network Configuration Assistant (INA) provides a policy-enforced framework for containerized network management. By combining natural-language intent parsing, FastMCP tool execution, an independent Policy Engine (`code/policy/policy_engine.py`), dynamic Docker SDK discovery, and kernel-level enforcement (`nftables` and bi-directional `tc tbf` / IFB shaping), INA reduces manual CLI syntax errors while enforcing policy compliance. The policy engine provides deterministic rule evaluation before privileged network operations, while empirical validation confirms that the tested operations converged to their expected network states.

References

- [1] Docker Inc., *Use bridge networks*, Docker Documentation, 2024. <https://docs.docker.com/network/drivers/bridge/>
- [2] Netfilter Core Team, *nftables HOWTO documentation*, 2024. <https://netfilter.org/projects/nftables/>
- [3] Linux Foundation, *tc(8) - Linux manual page*, iproute2, 2024. <https://man7.org/linux/man-pages/man8/tc.8.html>
- [4] Linux Kernel Organization, *Intermediate Functional Block (IFB) pseudo-device*, Linux Kernel Documentation, 2024.
- [5] ESnet, *iperf3: A tool for performing network throughput measurements*, 2024. <https://iperf.fr/>
- [6] Anthropic / Linux Foundation, *Model Context Protocol Specification*, 2024. <https://modelcontextprotocol.io/>
- [7] Colvin, S. et al., *Pydantic: Data validation using Python type hints*, 2024. <https://docs.pydantic.dev/>